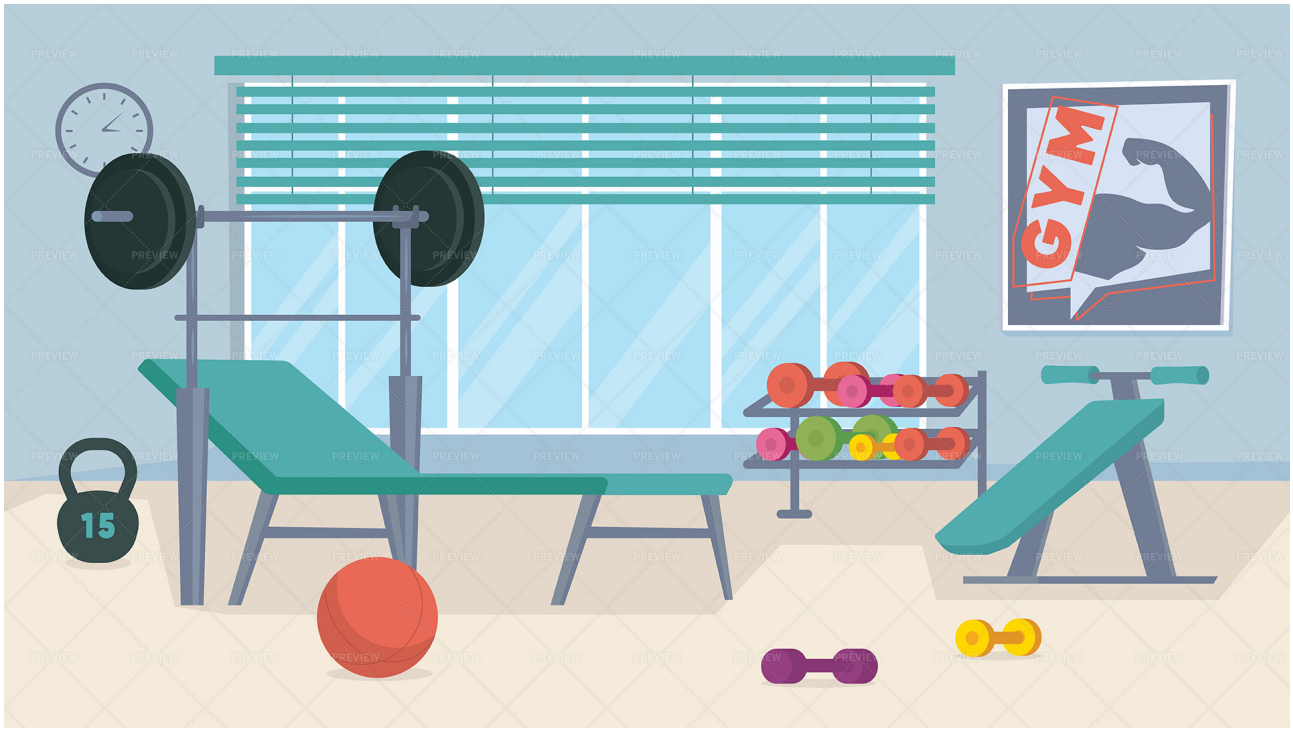
# PROJECT - PHASE 2
# GYM MANAGEMENT SYSTEM




**Students Group:**

**Leader: Razan Alkhathaami, ID: 445203318**
**Alanoud Aljohar, ID: 446206778**
**Raseel Abanmi, ID: 446201995**

**The Division of Work:**

**Razan: Files, Exceptions, and File Handling**
**Alanoud: Data Structure (Linked List) and Report**
**Raseel: Graphical User Interface (GUI)**

**Introduction:**
The program is a Gym Membership Management System, which allows the user to manage memberships and customer subscriptions in the gym. The system helps in adding new memberships, creating subscriptions, canceling subscriptions, searching for subscriptions, and viewing all available memberships according to their type and availability.

There are three types of memberships: Regular Memberships for basic gym access with a limited number of days per week, Premium Memberships which include extra features such as personal trainer and diet plan, and VIP Memberships which provide additional guest passes and premium services. Each membership has its own price according to the included features and services.

**Modified Code:**
1. Linked List Part: The main modification in Phase 2 was replacing the arrays used to store memberships and subscriptions with linked lists. In Phase 1, the system used Membership[] membershipList and Subscription[] subscriptions, which had a fixed size and depended on array indexing. In Phase 2, these were replaced with List membershipList and List subscriptions by creating two new classes: Node and List. The Node class stores the object and a reference to the next node, while the List class manages the linked list using head and tail references. Methods such as adding, deleting, searching, and viewing memberships were updated to work using nodes instead of array indexes. This made the system more flexible and improved data storage by allowing dynamic insertion and deletion without relying on fixed-size arrays.
2. Files and Exceptions Part: Phase 2 added file handling and exception handling. Memberships.dat and Subscriptions.dat were created using serialization to save data permanently. The methods saveAllInfo() and readAllData() were added in the Gym class for file writing and reading. Exception handling was added in TestGym using try-catch blocks and custom exceptions like InvalidNameException and InvalidPhoneException, along with InputMismatchException and ArithmeticException. This improved data safety and input validation.
3. GUI Part:

**How Exceptions Were Handled:**
We handled exceptions in different parts of the program to validate user input and prevent the system from crashing.
* InputMismatchException was handled in the main menu and in addNewSubscription() when the user enters invalid input instead of numbers.
* InvalidNameException was handled in addNewSubscription() and thrown inside readValidName() to ensure the name contains letters only.
* InvalidPhoneException was handled in addNewSubscription() and thrown inside readValidPhone() to validate the phone number format.
* ArithmeticException was handled in addNewSubscription() for invalid months input and in addNewMembership() for invalid price values.
* IllegalArgumentException was handled in addNewMembership() when the membership type entered is not between 1 and 3.

* NullPointerException was handled in searchSubscription() when the subscription does not exist.
* IOException was handled in saveAllInfo() and readAllData() during file operations.
* ClassNotFoundException was handled in readAllData() while reading serialized objects from files.

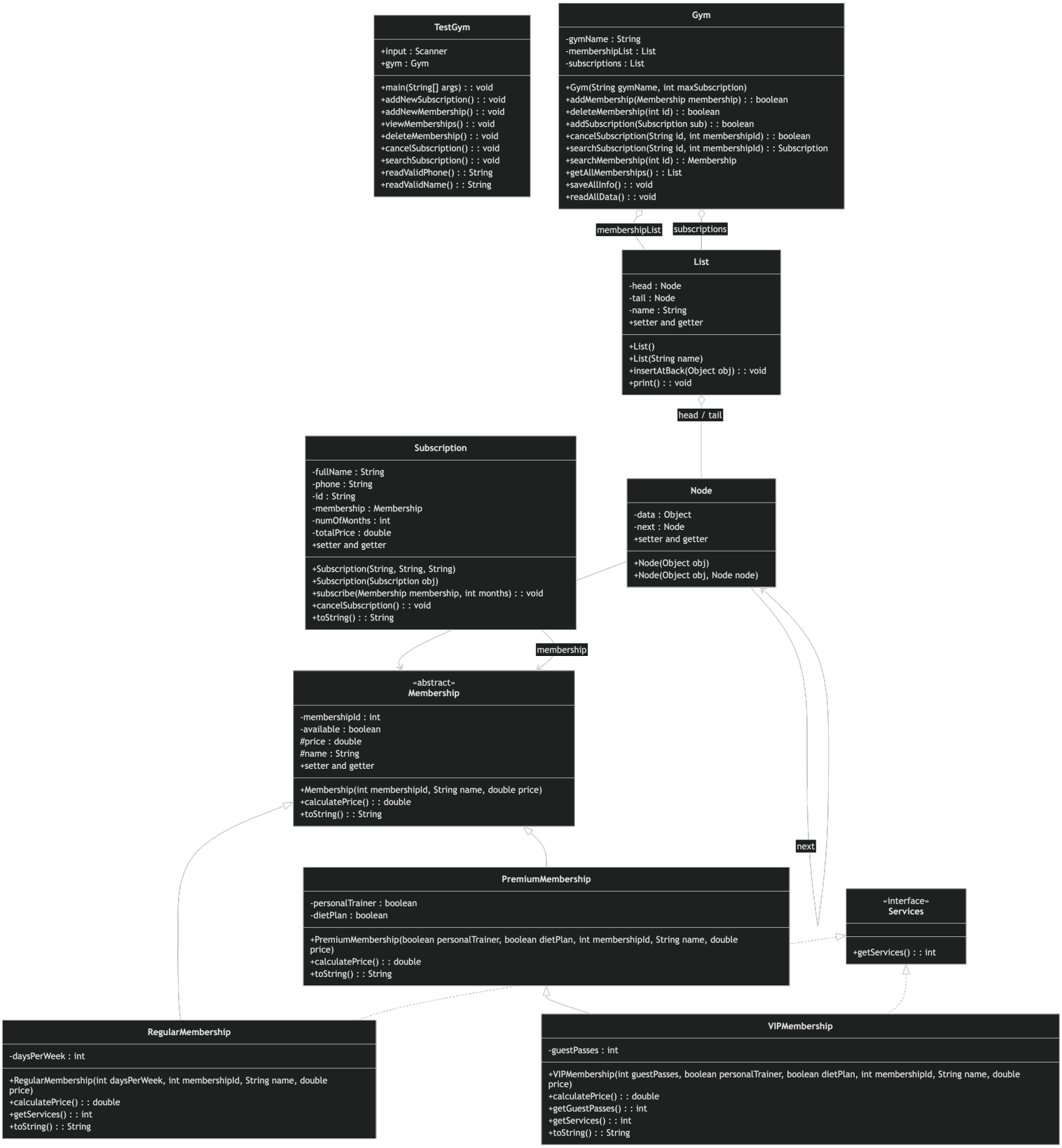
**How Files Were Managed:**
* Added implements Serializable to the following classes:
    * Gym
    * Membership
    * Subscription
    * Node
    * List

This modification allows objects to be serialized and stored in files so the program data can be saved and restored later.

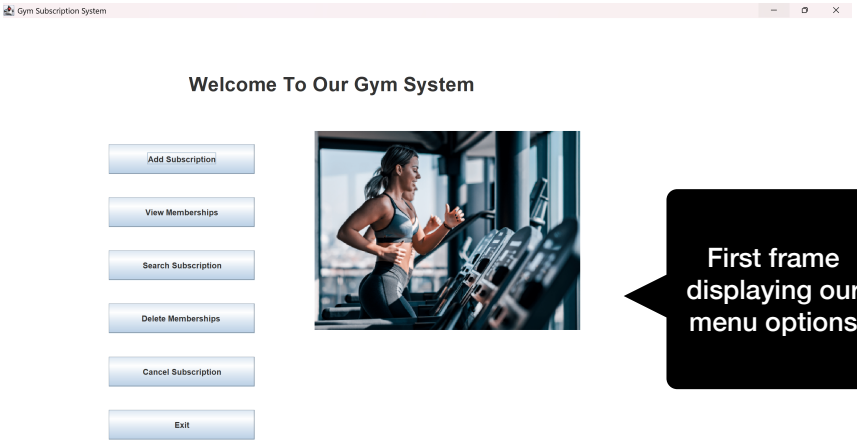
* In saveAllInfo() we used:
    * FileOutputStream
    * ObjectOutputStream
* to save memberships and subscriptions into:
    * Memberships.dat
    * Subscriptions.dat
* In readAllData() we used:
    * FileInputStream
    * ObjectInputStream
* to read saved objects from the files and restore the data when the program starts.
* Added gym.readAllData() in TestGym to load memberships and subscriptions automatically from files when the program starts.
* Added gym.saveAllInfo() at the end of:
    * addNewSubscription()
    * addNewMembership()
    * cancelSubscription()
    * deleteMembership()
* to ensure all changes are saved immediately and data is not lost even if the program is closed.

* We also used: if (f.exists() && f2.exists()) in main() to check if the files already exist before loading data.

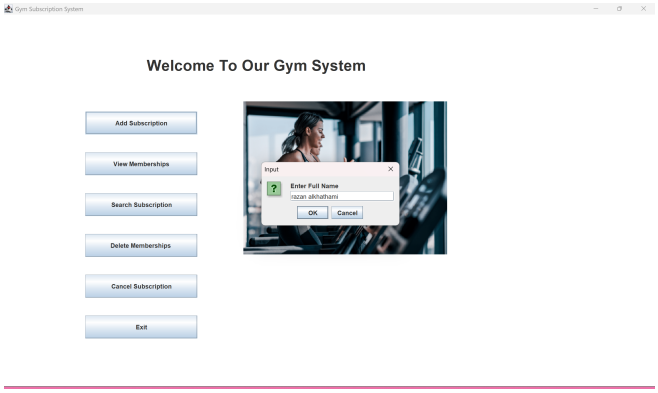
# UML

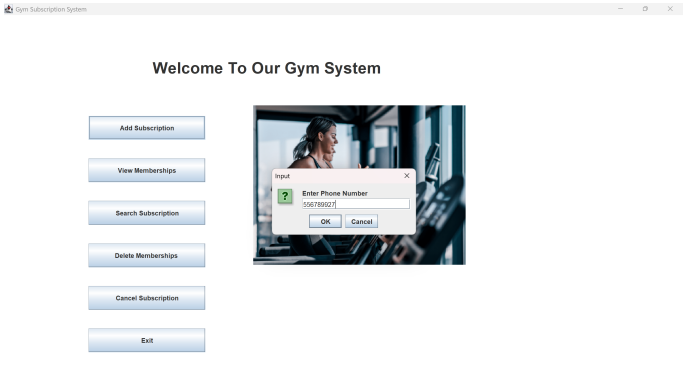

TestGym
+input : Scanner
+gym : Gym
+main(String[] args) : : void
+addNewSubscription() : : void
+addNewMembership() : : void
+viewMemberships() : : void
+deleteMembership() : : void
+cancelSubscription() : : void
+searchSubscription() : : void
+readValidPhone() : : String
+readValidName() : : String
Gym
-gymName : String
-membershipList : List
-subscriptions : List
+Gym(String gymName, int maxSubscription)
+addMembership(Membership membership) : : boolean
+deleteMembership(int id) : : boolean
+addSubscription(Subscription sub) : : boolean
+cancelSubscription(String id, int membershipId) : : boolean
+searchSubscription(String id, int membershipId) : : Subscription
+searchMembership(int id) : : Membership
+getAllMemberships() : : List
+saveAllInfo() : : void
+readAllData() : : void
membershipList
subscriptions
List
-head : Node
-tail : Node
-name : String
+setter and getter
+List()
+List(String name)
+insertAtBack(Object obj) : : void
+print() : : void
head / tail
Subscription
-fullName : String
-phone : String
-id : String
-membership : Membership
-numOfMonths : int
-totalPrice : double
+setter and getter
+Subscription(String, String, String)
+Subscription(Subscription obj)
+subscribe(Membership membership, int months) : : void
+cancelSubscription() : : void
+toString() : : String
Node
-data : Object
-next : Node
+setter and getter
+Node(Object obj)
+Node(Object obj, Node node)
membership
«abstract»
Membership
-membershipId : int
-available : boolean
#price : double
#name : String
+setter and getter
+Membership(int membershipId, String name, double price)
+calculatePrice() : : double
+toString() : : String
next
PremiumMembership
-personalTrainer : boolean
-dietPlan : boolean
+PremiumMembership(boolean personalTrainer, boolean dietPlan, int membershipId, String name, double price)
+calculatePrice() : : double
+toString() : : String
«interface»
Services
+getServices() : : int
RegularMembership
-daysPerWeek : int
+RegularMembership(int daysPerWeek, int membershipId, String name, double price)
+calculatePrice() : : double
+getServices() : : int
+toString() : : String
VIPMembership
-guestPasses : int
+VIPMembership(int guestPasses, boolean personalTrainer, boolean dietPlan, int membershipId, String name, double price)
+calculatePrice() : : double
+getGuestPasses() : : int
+getServices() : : int
+toString() : : String

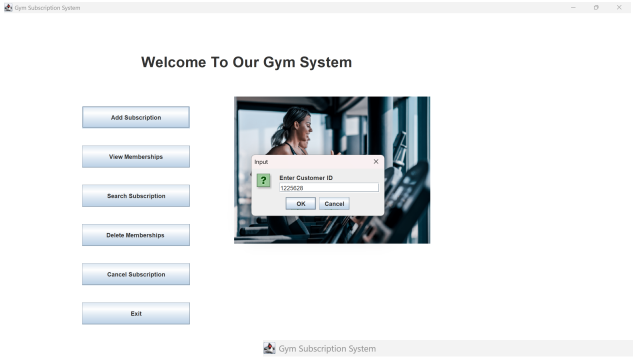
# GUI

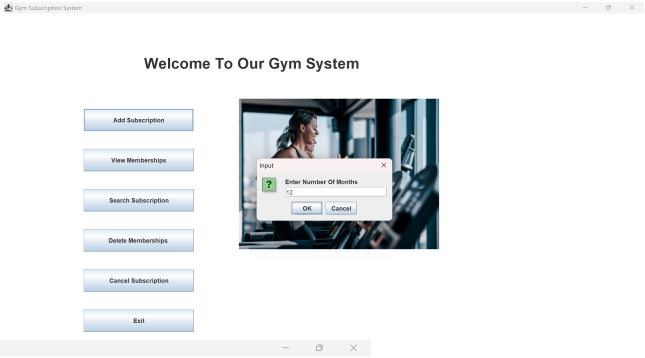


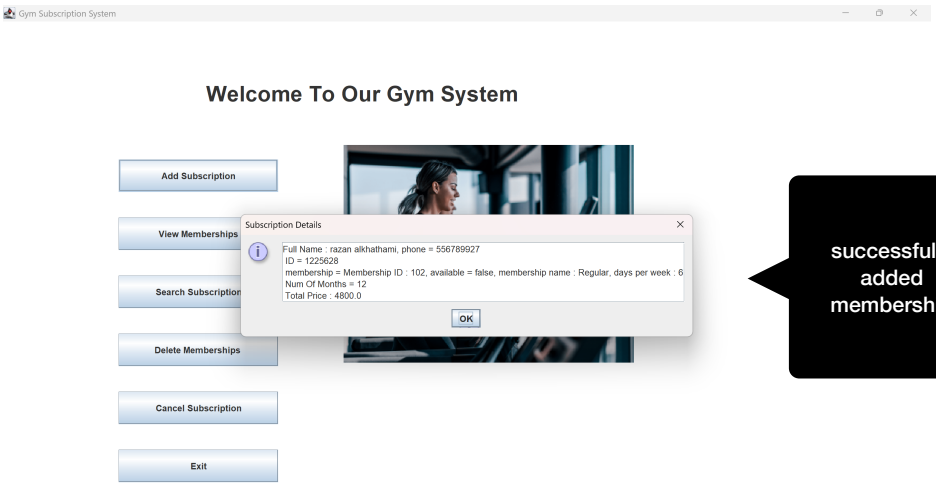

adding membership:



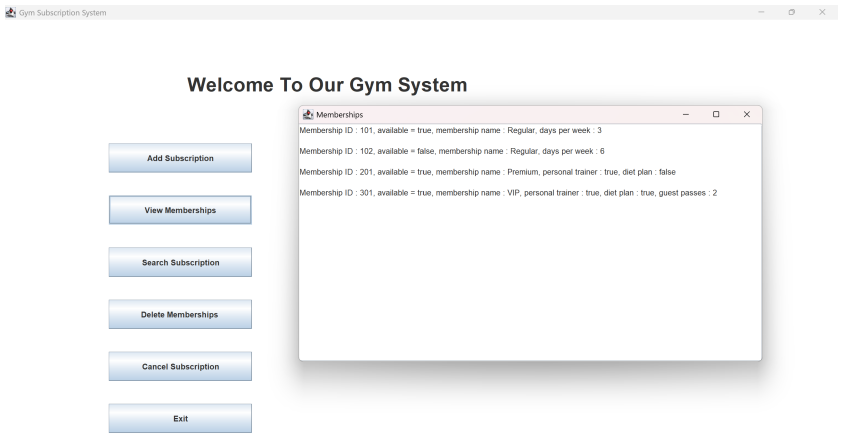



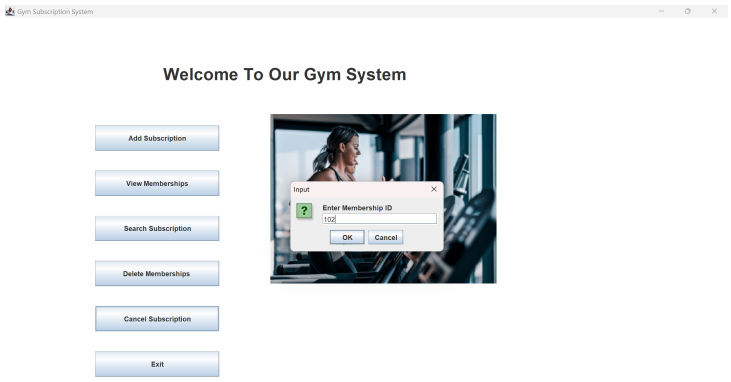



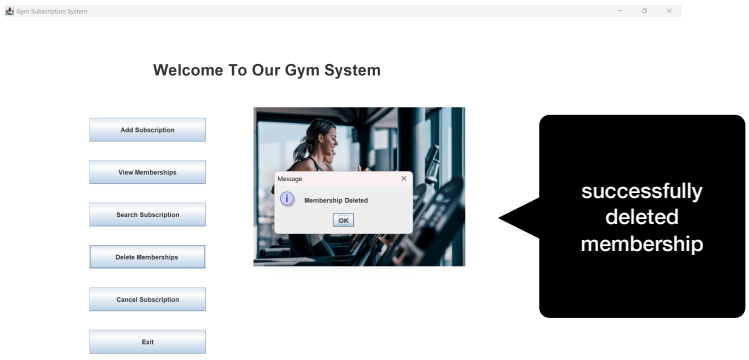



## view memberships:

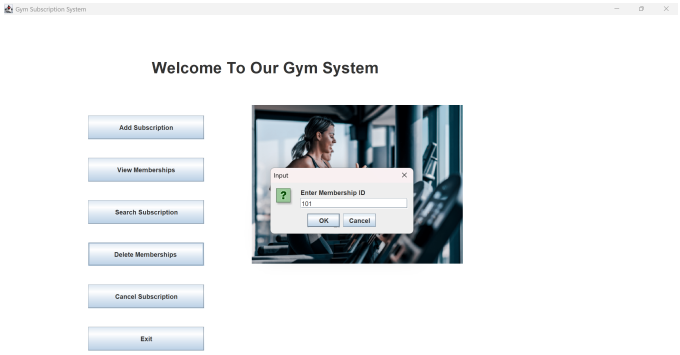


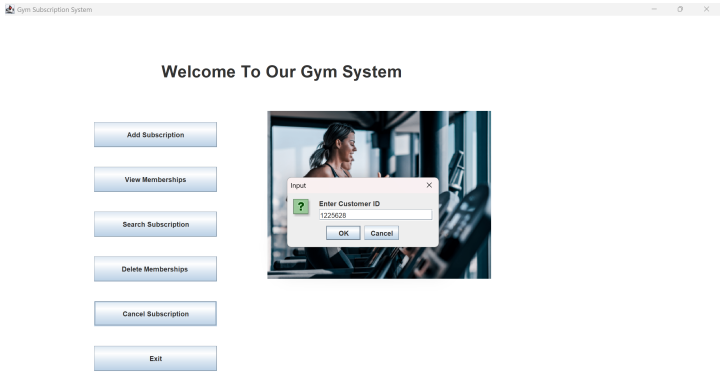

## delete membership:

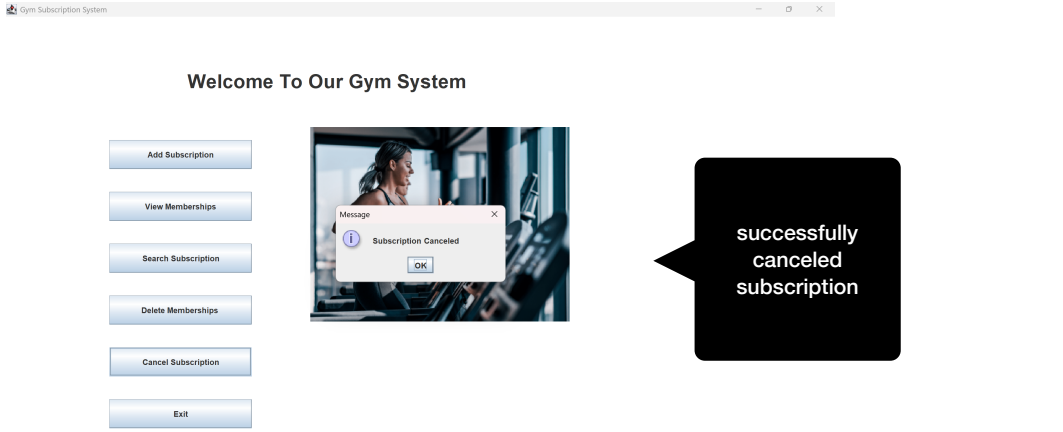


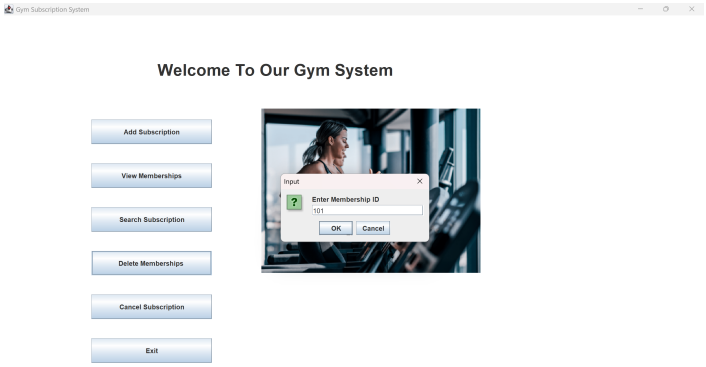

## cancel subscription:

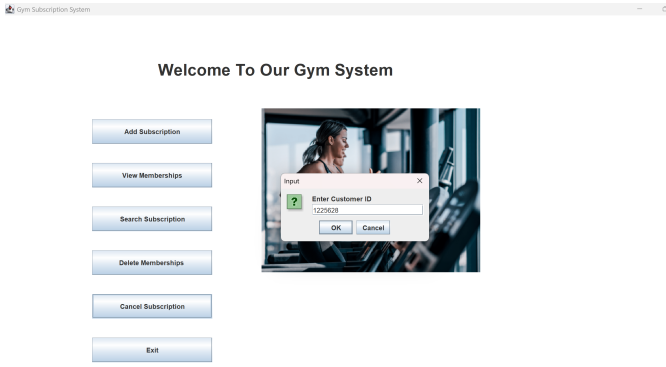


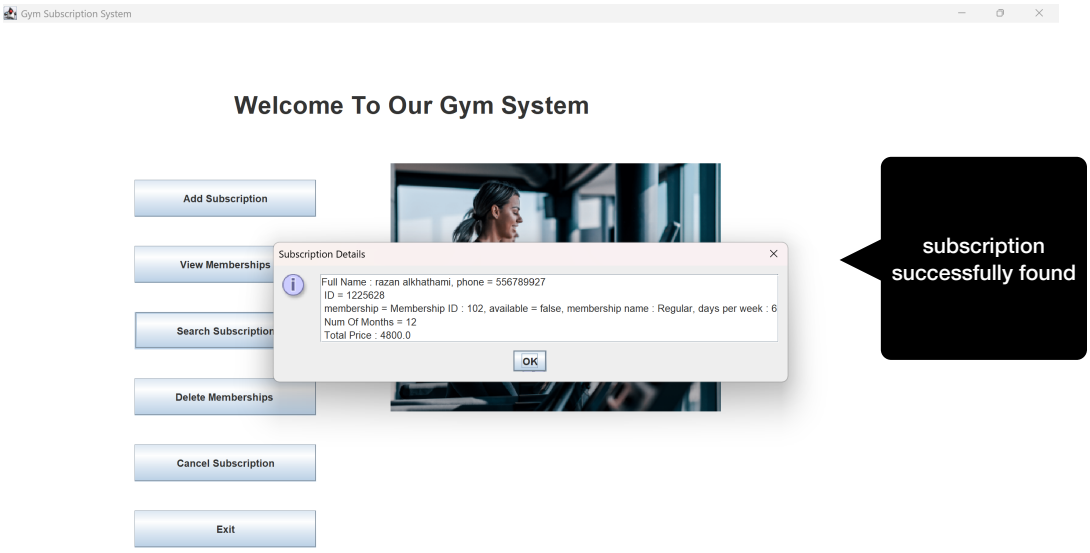



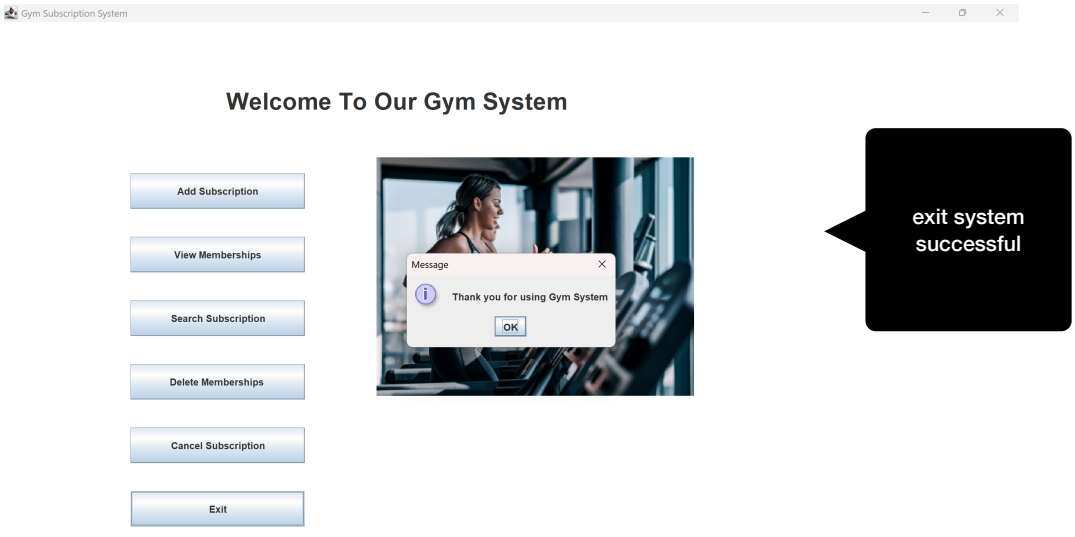
search subscription:




exit:



# Phase 2 Code

```java
public interface Services {
    int getServices();
}

////////////////////////////////////
```

```java
import java.io.Serializable;

public abstract class Membership implements Serializable{
    private int membershipId;
    private boolean available;
    protected double price;
    protected String name;

    // constructor
    public Membership(int membershipId, String name, double price) {
        this.membershipId = membershipId;
        this.name = name;
        this.price = price;
        this.available = true;
    }

    // abstract method for price calculation
    public abstract double calculatePrice();

    public int getMembershipId() {
        return membershipId;
    }

    public void setMembershipId(int membershipId) {
        this.membershipId = membershipId;
    }

    public boolean isAvailable() {
        return available;
    }

    public void setAvailable(boolean available) {
        this.available = available;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public String toString() {
        return "Membership ID : " + membershipId + ", available = " + available +
               ", membership name : " + name;
```

```
    }
}

//////////////////////////////////
```

```
public class Node {
   private Object data;
   private Node next;

   public Node(Object obj) {
      data = obj;
      next = null;
   }

   public Node(Object obj, Node node) {
      data = obj;
      next = node;
   }

   public Object getData() {
      return data;
   }

   public Node getNext() {
      return next;
   }

   public void setNext(Node next) {
      this.next = next;
   }

   public String toString() {
      return data.toString();
   }
}

//////////////////////////////////
```

```
public class List {
   private Node head;
   private Node tail;
   private String name;

   public List() {
      head = tail = null;
      name = "No Name";
   }

   public List(String name) {
      head = tail = null;
      this.name = name;
   }

   public boolean isEmpty() {
      return head == null;
   }

   public void insertAtBack(Object obj) {
      Node newnode = new Node(obj);

      if (isEmpty())
```

```
            head = tail = newnode;
        else {
            tail.setNext(newnode);
            tail = newnode;
        }
    }

    public Node getHead() {
        return head;
    }

    public Node getTail() {
        return tail;
    }

    public void setHead(Node head) {
        this.head = head;
    }

    public void setTail(Node tail) {
        this.tail = tail;
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("The list " + name + " is empty");
            return;
        }

        Node current = head;

        while (current != null) {
            System.out.println(current.getData());
            current = current.getNext();
        }
    }
}

////////////////////////////////////
```

```
public class RegularMembership extends Membership implements Services {
    private int daysPerWeek;

    // constructor
    public RegularMembership(int daysPerWeek, int membershipId, String name, double price) {
        super(membershipId, name, price);
        this.daysPerWeek = daysPerWeek;
    }

    // calculate price based on days
    public double calculatePrice() {
        double total = price;

        if (daysPerWeek >= 5)
            total = price + 100;

        return total;
    }

    // return number of services
    public int getServices() {
```

```
        return 1;
    }

    public String toString() {
        return super.toString() + ", days per week : " + daysPerWeek;
    }
}

////////////////////////////////////
```

```
public class PremiumMembership extends Membership {
    private boolean personalTrainer;
    private boolean dietPlan;

    // constructor
    public PremiumMembership(boolean personalTrainer, boolean dietPlan, int membershipId, String
name, double price) {
        super(membershipId, name, price);
        this.personalTrainer = personalTrainer;
        this.dietPlan = dietPlan;
    }

    // calculate price with extra features
    public double calculatePrice() {
        double total = price;

        if (personalTrainer)
            total = total + 200;

        if (dietPlan)
            total = total + 150;

        return total;
    }

    public String toString() {
        return super.toString() + ", personal trainer : " + personalTrainer + ", diet plan : " +
dietPlan;
    }
}

////////////////////////////////////
```

```
public class VIPMembership extends PremiumMembership implements Services {
   private int guestPasses;

   // constructor
   public VIPMembership(int guestPasses, boolean personalTrainer, boolean dietPlan,
                        int membershipId, String name, double price) {
      super(personalTrainer, dietPlan, membershipId, name, price);
      this.guestPasses = guestPasses;
   }

   // calculate price including guest passes
   public double calculatePrice() {
      double total = super.calculatePrice();
      total = total + (guestPasses * 50);
      return total;
   }

   // return number of guest passes
```

```
    public int getGuestPasses() {
       return guestPasses;
    }

    // return number of services
    public int getServices() {
       return guestPasses + 2;
    }

    public String toString() {
       return super.toString() + ", guest passes : " + guestPasses;
    }
}

////////////////////////////////////
```

```
import java.io.Serializable;

public class Subscription implements Serializable {
    private String fullName;
    private String phone;
    private String id;
    private Membership membership;
    private int numOfMonths;
    private double totalPrice;

    // constructor
    public Subscription(String fullName, String phone, String id) {
        this.fullName = fullName;
        this.phone = phone;
        this.id = id;
    }


public Subscription(
    String fullName,
    String phone,
    String id,
    Membership membership,
    int months)
{
    this.fullName = fullName;
    this.phone = phone;
    this.id = id;

    this.membership = membership;
    this.numOfMonths = months;

    membership.setAvailable(false);

    totalPrice =
        numOfMonths *
        membership.calculatePrice();
}

    // copy constructor
    public Subscription(Subscription obj) {
        this.fullName = obj.fullName;
        this.phone = obj.phone;
        this.id = obj.id;
        this.numOfMonths = obj.numOfMonths;
```

```
        this.membership = obj.membership;
        this.totalPrice = obj.totalPrice;
    }

    // subscribe to membership
    public void subscribe(Membership membership, int months) {
        numOfMonths = months;
        this.membership = membership;
        membership.setAvailable(false);
        totalPrice = numOfMonths * membership.calculatePrice();
    }

    // cancel subscription
    public void cancelSubscription() {
        if (membership.isAvailable())
            System.out.println("This subscription was already canceled.");
        else {
            membership.setAvailable(true);
            System.out.println("Subscription is canceled.");
        }
    }

    public String toString() {
        return "Full Name : " + fullName + ", phone = " + phone +
               "\n ID = " + id + "\n membership = " + membership +
               "\n Num Of Months = " + numOfMonths +
               "\n Total Price : " + totalPrice;
    }

    public String getID() {
        return id;
    }

    public Membership getMembership() {
        return membership;
    }
}

////////////////////////////////////
```

```
import java.io.*;

public class Gym implements Serializable{
    private String gymName;
    List membershipList;
    List subscriptions;

    // constructor
    public Gym(String gymName, int maxSubscription) {
       this.gymName = gymName;
       membershipList = new List("membership list");
       subscriptions = new List("subscription list");
    }

    // add membership
    public boolean addMembership(Membership membership) {
       membershipList.insertAtBack(membership);
       return true;
    }

    // delete membership
```

```
22    public boolean deleteMembership(int id) {
23       Node current = membershipList.getHead();
24       Node previous = null;
25
26       while (current != null) {
27          Membership m = (Membership) current.getData();
28
29          if (m.getMembershipId() == id) {
30             if (m.isAvailable()) {
31
32                if (previous == null)
33                   membershipList.setHead(current.getNext());
34                else
35                   previous.setNext(current.getNext());
36
37                if (current == membershipList.getTail())
38                   membershipList.setTail(previous);
39
40                return true;
41             }
42             else {
43                System.out.println("This membership is already taken by another person");
44                return false;
45             }
46          }
47
48          previous = current;
49          current = current.getNext();
50       }
51
52       return false;
53    }
54
55    // add subscription
56    public boolean addSubscription(Subscription sub) {
57       subscriptions.insertAtBack(new Subscription(sub));
58       return true;
59    }
60
61    // cancel subscription
62    public boolean cancelSubscription(String id, int membershipId) {
63       Node current = subscriptions.getHead();
64       Node previous = null;
65
66       while (current != null) {
67          Subscription sub = (Subscription) current.getData();
68
69          if (sub.getID().equals(id) &&
70              sub.getMembership().getMembershipId() == membershipId) {
71
72             sub.cancelSubscription();
73
74             if (previous == null)
75                subscriptions.setHead(current.getNext());
76             else
77                previous.setNext(current.getNext());
78
79             if (current == subscriptions.getTail())
80                subscriptions.setTail(previous);
81
```

```
            return true;
        }

        previous = current;
        current = current.getNext();
    }

    return false;
}

// search subscription
public Subscription searchSubscription(String id, int membershipId) {
    Node current = subscriptions.getHead();

    while (current != null) {
        Subscription sub = (Subscription) current.getData();

        if (sub.getID().equals(id) &&
             sub.getMembership().getMembershipId() == membershipId)
            return sub;

        current = current.getNext();
    }

    return null;
}

// search membership
public Membership searchMembership(int id) {
    Node current = membershipList.getHead();

    while (current != null) {
        Membership m = (Membership) current.getData();

        if (m.getMembershipId() == id)
            return m;

        current = current.getNext();
    }

    return null;
}

// get all memberships
public List getAllMemberships() {
    return membershipList;
}


// SAVE ALL DATA

public void saveAllInfo() {

    try {

        // save memberships
        File out = new File("Memberships.dat");

        FileOutputStream fos = new FileOutputStream(out);

```

```
142         ObjectOutputStream oos =
143               new ObjectOutputStream(fos);
144
145         oos.writeObject(membershipList);
146
147         oos.close();
148
149         // save subscriptions
150         File out2 = new File("Subscriptions.dat");
151
152         FileOutputStream fos2 =
153               new FileOutputStream(out2);
154
155         ObjectOutputStream oos2 =
156               new ObjectOutputStream(fos2);
157
158         oos2.writeObject(subscriptions);
159
160         oos2.close();
161
162      }
163
164      catch (IOException e) {
165
166         System.out.println(e.toString());
167
168      }
169   }
170
171
172   // READ ALL DATA
173
174
175   public void readAllData() {
176
177      try {
178
179         // read memberships
180         File f = new File("Memberships.dat");
181
182         FileInputStream ff =
183               new FileInputStream(f);
184
185         ObjectInputStream in =
186               new ObjectInputStream(ff);
187
188         membershipList = (List) in.readObject();
189
190         in.close();
191
192         // read subscriptions
193         File f2 = new File("Subscriptions.dat");
194
195         FileInputStream ff2 =
196               new FileInputStream(f2);
197
198         ObjectInputStream in2 =
199               new ObjectInputStream(ff2);
200
201         subscriptions =
```

```
202                (List) in2.readObject();
203
204          in2.close();
205
206          System.out.println(
207                "All data in files are loaded.");
208
209       }
210
211       catch (ClassNotFoundException ex) {
212
213          System.out.println(ex.toString());
214
215       }
216
217       catch (IOException e) {
218
219          System.out.println(e.toString());
220
221       }
222
223 }
224
225 }
226
227 ////////////////////////////////////
```

```
1 public class InvalidNameException extends Exception {
2
3    InvalidNameException(String s) {
4
5       super(s);
6    }
7 }
8
9 ////////////////////////////////////
```

```
1 public class InvalidPhoneException extends Exception {
2
3    InvalidPhoneException(String s) {
4
5       super(s);
6    }
7 }
8
9 ////////////////////////////////////
```

```
 1 import java.util.*;
 2 import java.io.*;
 3
 4 public class TestGym {
 5
 6    static Scanner input = new Scanner(System.in);
 7    static Gym gym = new Gym("Power Gym", 1000);
 8
 9 public static void main(String[] args) {
10
11       File f = new File("Memberships.dat");
12
13       File f2 = new File("Subscriptions.dat");
14
```

```
15      if (f.exists() && f2.exists()) {
16
17         gym.readAllData();}
18
19      else{
20      // create memberships
21      RegularMembership reg1 =
22         new RegularMembership(3, 101, "Regular", 250);
23
24      RegularMembership reg2 =
25         new RegularMembership(6, 102, "Regular", 300);
26
27      PremiumMembership prem1 =
28         new PremiumMembership(true, false,201, "Premium", 500);
29
30      PremiumMembership prem2 =
31         new PremiumMembership(true, true, 202, "Premium", 550);
32
33      VIPMembership vip1 =
34         new VIPMembership(2, true, true, 301, "VIP", 800);
35
36      VIPMembership vip2 =
37         new VIPMembership(4, true, true, 302, "VIP", 950);
38
39      // add to gym
40      gym.addMembership(reg1);
41      gym.addMembership(reg2);
42      gym.addMembership(prem1);
43      gym.addMembership(prem2);
44      gym.addMembership(vip1);
45      gym.addMembership(vip2);
46    }
47
48      int choice = 0;
49
50   // menu starts here
51        do {
52
53
54         try {
55
56         System.out.println("***** Menu *****");
57         System.out.println("1- Add new subscription");
58         System.out.println("2- View memberships");
59         System.out.println("3- Cancel subscription");
60         System.out.println("4- Search subscription");
61         System.out.println("5- Add new membership");
62         System.out.println("6- Delete membership");
63         System.out.println("7- Exit");
64
65         choice = input.nextInt();
66
67         switch (choice) {
68            case 1:
69               addNewSubscription();
70               break;
71            case 2:
72               viewMemberships();
73               break;
74            case 3:
```

```
75                cancelSubscription();
76                break;
77             case 4:
78                searchSubscription();
79                break;
80             case 5:
81                addNewMembership();
82                break;
83             case 6:
84                deleteMembership();
85                break;
86             case 7:
87                System.out.println("**** Goodbye! *****");
88                break;
89             default:
90                System.out.println("Invalid input");
91          }
92
93          }catch(InputMismatchException e) {
94
95          System.out.println("Enter digits only");
96          input.next();}
97
98
99       } while(choice != 7);
100   }
101
102   // add subscription
103   public static void addNewSubscription() {
104      System.out.println("Enter membership ID: ");
105
106      int id = 0;
107
108      try {
109
110         id = input.nextInt();
111
112      }catch(InputMismatchException e) {
113
114         System.out.println("Enter digits only");
115
116         input.next();
117
118         return;}
119
120           Membership membershipObj = gym.searchMembership(id);
121
122      if (membershipObj == null || membershipObj.isAvailable() == false) {
123         System.out.println("This membership is not available, try again");
124         return;
125      }
126
127      input.nextLine();
128
129      String name = "";
130      String phone = "";
131
132      try {
133
134         name = readValidName();
```

```
135          phone = readValidPhone();
136
137       }catch (InvalidNameException e) {
138
139          System.out.println(e.getMessage());
140          return;}
141
142        catch (InvalidPhoneException e) {
143
144          System.out.println(e.getMessage());
145          return; }
146
147       System.out.println("Enter customer ID: ");
148       String idNum = input.next();
149
150       Subscription sub = new Subscription(name, phone, idNum);
151
152       int months = 0;
153
154       try {
155
156          System.out.println("Enter number of months:");
157
158          months = input.nextInt();
159
160       if (months <= 0)throw new ArithmeticException("Months must be positive");
161
162          }catch (ArithmeticException e) {
163
164          System.out.println(e.getMessage());
165
166          return;}
167
168       sub.subscribe(membershipObj, months);
169
170       gym.addSubscription(sub);
171
172       gym.saveAllInfo();
173
174       System.out.println("Subscription has been added successfully");
175       System.out.println(sub);
176
177    }
178
179    // add membership
180    public static void addNewMembership() {
181       System.out.println("Enter what type you want the membership to be : 1-regular , 2-premium
3- vip");
182       int type = input.nextInt();
183
184       System.out.println("Enter an ID to create a new membership");
185       int addid = input.nextInt();
186
187       System.out.println("Enter a name for the membership ");
188       input.nextLine();
189       String membership_name = input.nextLine();
190
191       double price = 0;
192
193       try {
```

```
194
195       System.out.println("Enter membership price:");
196
197       price = input.nextDouble();
198
199       if (price <= 0)
200
201       throw new ArithmeticException("Price must be positive");
202
203
204       }catch (ArithmeticException e) {
205
206       System.out.println(e.getMessage());
207       return;}
208
209       Membership new_membership = null;
210
211       try {
212
213       if (type < 1 || type > 3) throw new IllegalArgumentException("Invalid membership type");
214
215       }catch (IllegalArgumentException e) {
216        System.out.println(e.getMessage());
217        return;}
218
219       if (type == 1) {
220          System.out.println("how many days per week the person can come to the gym in this
membership?");
221          int daysperweek = input.nextInt();
222          new_membership = new RegularMembership(daysperweek, addid, membership_name, price);
223       }
224       else if (type == 2) {
225          System.out.println("do you want this membership to have personal trainer ?\nplease
chose true or false");
226          boolean personaltranier = input.nextBoolean();
227          System.out.println("do you want this membership to have a customised diet plan ?
\nplease chose true or false");
228          boolean dietplan = input.nextBoolean();
229
230          new_membership = new PremiumMembership(personaltranier, dietplan, addid,
membership_name, price);
231       }
232       else if (type == 3) {
233          System.out.println("how many guest passes you want in this membership?");
234          int guestpasses = input.nextInt();
235          System.out.println("do you want this membership to have personal trainer ?\nplease
chose true or false");
236          boolean personaltranier = input.nextBoolean();
237          System.out.println("do you want this membership to have a customised diet plan ?
\nplease chose true or false");
238          boolean dietplan = input.nextBoolean();
239
240          new_membership = new VIPMembership(guestpasses, personaltranier, dietplan, addid,
membership_name, price);
241       }
242
243         if (new_membership != null) {
244
245          gym.addMembership(new_membership);
246
```

```
247          gym.saveAllInfo();
248
249          System.out.println("addition has been done\n");
250       }
251       else
252          System.out.println("Can't add\n");    }
253
254    // view memberships
255    public static void viewMemberships() {
256       List list = gym.getAllMemberships();
257       list.print();
258    }
259
260
261
262    // delete membership
263    public static void deleteMembership() {
264
265    System.out.println("Enter membership ID");
266    int id = input.nextInt();
267
268    if (gym.deleteMembership(id)) {
269
270       gym.saveAllInfo();
271
272       System.out.println("Deletion has been done\n");
273    }
274    else
275       System.out.println("Can't delete\n");
276 }
277
278    // cancel subscription
279    public static void cancelSubscription() {
280       System.out.println("Enter customer ID: ");
281       String id = input.next();
282       System.out.println("Enter membership ID: ");
283       int no = input.nextInt();
284
285       if (gym.cancelSubscription(id, no)) {
286
287       gym.saveAllInfo();
288
289       System.out.println("Cancellation has been done.");
290       }
291       else
292       System.out.println("Sorry, can't cancel.");
293    }
294
295    // search subscription
296    public static void searchSubscription() {
297       System.out.println("Enter customer ID: ");
298       String id = input.next();
299       System.out.println("Enter membership ID: ");
300       int no = input.nextInt();
301
302       try {
303
304          Subscription sub = gym.searchSubscription(id, no);
305
306          if (sub == null)
```

```
307
308          throw new NullPointerException("Subscription not found");
309
310          System.out.println(sub);
311
312          }catch (NullPointerException e) {
313          System.out.println(e.getMessage());}
314
315          }
316
317 public static String readValidPhone()
318        throws InvalidPhoneException {
319
320    String phone;
321
322    System.out.print("Enter phone number: ");
323    phone = input.next();
324
325    if (phone.length() != 10)
326       throw new InvalidPhoneException(
327             "Phone must be 10 digits");
328
329    for (int i = 0; i < phone.length(); i++) {
330
331       if (!Character.isDigit(phone.charAt(i)))
332          throw new InvalidPhoneException(
333                "Phone must contain digits only");
334    }
335
336    return phone;
337 }
338
339 public static String readValidName()
340        throws InvalidNameException {
341
342    String name;
343
344    System.out.print("Enter full name: ");
345    name = input.nextLine();
346
347    for (int i = 0; i < name.length(); i++) {
348
349       char ch = name.charAt(i);
350
351       if (!Character.isLetter(ch) && ch != ' ')
352          throw new InvalidNameException(
353                "Name must contain letters only");
354    }
355
356    return name;
357 }
358
359 }
360
361 ////////////////////////////////////
```

```
1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.*;
4 import java.io.File;
5
```

```java
public class GymGUI extends JFrame implements ActionListener {

    Gym gym;

   JButton customerButton;
JButton membershipsButton;
JButton addMembershipButton;
JButton deleteMembershipButton;
JButton cancelSubscriptionButton;
JButton searchButton;
JButton deleteButton;
JButton cancelButton;
JButton exitButton;


    JLabel titleLabel;
    JLabel imageLabel;

    public GymGUI() {

        gym = new Gym("Power Gym", 1000);

        File f = new File("Memberships.dat");
        File f2 = new File("Subscriptions.dat");

        if (f.exists() && f2.exists()) {

            gym.readAllData();
        }

        else {

            gym.addMembership(
                    new RegularMembership(3,101,
                            "Regular",250));

            gym.addMembership(
                    new RegularMembership(6,102,
                            "Regular",300));

            gym.addMembership(
                    new PremiumMembership(true,false,
                            201,"Premium",500));

            gym.addMembership(
                    new VIPMembership(2,true,
                            true,301,"VIP",800));
        }

        setTitle("Gym Subscription System");

        setSize(1250,700);

        setLayout(null);

        getContentPane().setBackground(Color.white);

        titleLabel =new JLabel("Welcome To Our Gym System");

        titleLabel.setBounds(280,50,1200,100);
```

```
66
67          titleLabel.setFont(new Font("Arial",Font.BOLD,30));
68
69          add(titleLabel);
70
71          customerButton = new JButton("Add Subscription");
72
73          customerButton.setBounds(160,190,220,45);
74
75          membershipsButton = new JButton("View Memberships");
76
77          membershipsButton.setBounds(160,270,220,45);
78  searchButton = new JButton("Search Subscription");
79  deleteButton = new JButton("Delete Memberships ");
80  cancelButton = new JButton("Cancel Subscription");
81          exitButton = new JButton("Exit");
82
83  searchButton.setBounds(160,350,220,45);
84  deleteButton.setBounds(160,430,220,45);
85  cancelButton.setBounds(160,510,220,45);
86          exitButton.setBounds(160,590,220,45);
87
88
89          add(customerButton);
90          add(membershipsButton);
91          add(searchButton);
92          add(deleteButton);
93  add(cancelButton);
94
95          add(exitButton);
96
97
98          customerButton.addActionListener(this);
99
100         membershipsButton.addActionListener(this);
101 searchButton.addActionListener(this);
102 deleteButton.addActionListener(this);
103 cancelButton.addActionListener(this);
104
105         exitButton.addActionListener(this);
106
107          ImageIcon icon = new ImageIcon("gym-ladies-banner.png");
108
109          Image img = icon.getImage();
110
111          Image scaledImg =img.getScaledInstance(400,300,Image.SCALE_SMOOTH);
112
113          ImageIcon gymImage = new ImageIcon(scaledImg);
114
115          imageLabel = new JLabel(gymImage);
116
117          imageLabel.setBounds(470,170,400,300);
118
119          add(imageLabel);
120
121         setVisible(true);
122
123         setDefaultCloseOperation(
124                 JFrame.EXIT_ON_CLOSE);
125     }
```

```
126
127
128
129 public void actionPerformed(ActionEvent e) {
130
131     // ADD SUBSCRIPTION
132 if (e.getSource() == customerButton) {
133
134     String membershipId =
135         JOptionPane.showInputDialog(
136         "Enter Membership ID");
137
138     if (membershipId == null ||
139         membershipId.isEmpty()) {
140         return;
141     }
142
143     Membership membership =
144         gym.searchMembership(
145         Integer.parseInt(membershipId));
146
147     if (membership == null ||
148         !membership.isAvailable()) {
149
150         JOptionPane.showMessageDialog(
151             this,
152             "Membership Not Available");
153
154         return;
155     }
156
157     String name =
158         JOptionPane.showInputDialog(
159         "Enter Full Name");
160
161     String phone =
162         JOptionPane.showInputDialog(
163         "Enter Phone Number");
164
165     String customerId =
166         JOptionPane.showInputDialog(
167         "Enter Customer ID");
168
169     String monthsText =
170         JOptionPane.showInputDialog(
171         "Enter Number Of Months");
172
173     int months =
174         Integer.parseInt(monthsText);
175
176     Subscription sub =
177         new Subscription(
178             name,
179             phone,
180             customerId,
181             membership,
182             months);
183
184     gym.addSubscription(sub);
185
```

```
    gym.saveAllInfo();

    JTextArea resultarea =
        new JTextArea(sub.toString());

    resultarea.setEditable(false);

    JOptionPane.showMessageDialog(
        this,
        new JScrollPane(resultarea),
        "Subscription Details",
        JOptionPane.INFORMATION_MESSAGE);
}
    // VIEW MEMBERSHIPS
    if (e.getSource() == membershipsButton) {

        JFrame frame =
    new JFrame("Memberships");

JTextArea area =
    new JTextArea();

List list =
    gym.getAllMemberships();

Node current =
    list.getHead();

while(current != null) {

    area.append(
        current.getData().toString()
        + "\n\n");

    current =
        current.getNext();
}

frame.add(new JScrollPane(area));

frame.setSize(400,400);

frame.setVisible(true);
}


if (e.getSource() == searchButton) {

    String serchcustomerId =
    JOptionPane.showInputDialog(
    "Enter Customer ID");

String serchmembershipId =
    JOptionPane.showInputDialog(
    "Enter Membership ID");

Subscription foundsub =
    gym.searchSubscription(
        serchcustomerId,
        Integer.parseInt(serchmembershipId));
```

```
246
247 if (foundsub == null) {
248
249     JOptionPane.showMessageDialog(
250         this,
251         "Subscription Not Found");
252
253 }
254 else {
255
256     JTextArea searchtarea =
257         new JTextArea(foundsub.toString());
258
259     searchtarea.setEditable(false);
260
261     JOptionPane.showMessageDialog(
262         this,
263         new JScrollPane(searchtarea),
264         "Subscription Details",
265         JOptionPane.INFORMATION_MESSAGE);
266 }
267
268
269 }
270
271
272
273 if (e.getSource() == deleteButton) {
274
275     String deletemembershipId =
276     JOptionPane.showInputDialog(
277     "Enter Membership ID");
278
279 boolean deleted =
280     gym.deleteMembership(
281         Integer.parseInt(deletemembershipId));
282
283 if (deleted) {
284
285     JOptionPane.showMessageDialog(
286         this,
287         "Membership Deleted");
288
289 } else {
290
291     JOptionPane.showMessageDialog(
292         this,
293         "Cannot Delete Membership");
294 }
295 }
296
297
298 if (e.getSource() == cancelButton) {
299 String cancelcustomerId =
300     JOptionPane.showInputDialog(
301     "Enter Customer ID");
302
303 String cancelmembershipId =
304     JOptionPane.showInputDialog(
305     "Enter Membership ID");
```

```
306
307 boolean canceled =
308     gym.cancelSubscription(
309     cancelcustomerId,
310     Integer.parseInt(cancelmembershipId));
311
312 if (canceled) {
313
314     JOptionPane.showMessageDialog(
315         this,
316         "Subscription Canceled");
317
318 } else {
319
320     JOptionPane.showMessageDialog(
321         this,
322         "Subscription Not Found");
323 }
324
325 }
326
327
328
329     // EXIT
330     if (e.getSource() == exitButton) {
331 JOptionPane.showMessageDialog(
332         this,
333         "Thank you for using Gym System");
334         System.exit(0);
335     }
336
337
338 }
339 public static void main(String[] args) {
340
341         new GymGUI();
342     }
343 }
344
345 ////////////////////////////////////
```